Solution: Validate Binary Search Tree

Problem Statement

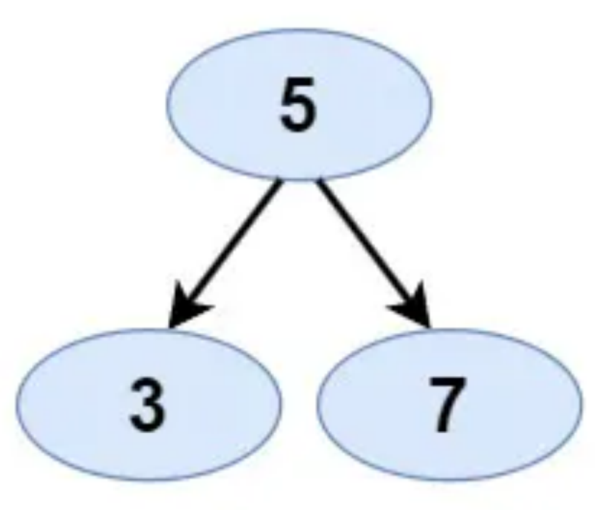
Determine if a given binary tree is a binary search tree (BST). In a BST, for each node:

- All nodes to its left have values less than the node's value.
- All nodes to its right have values greater than the node's value.

Example Generation

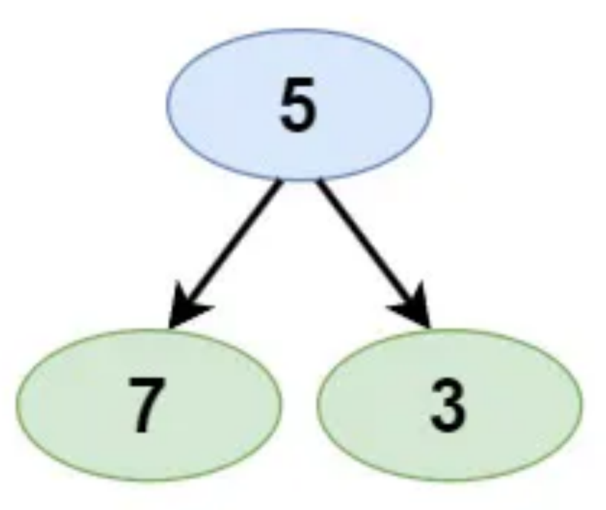
Example 1:

- **Input**: [5,3,7]
- **Expected Output**: true
- **Justification**: The left child of the root (3) is less than the root, and the right child of the root (7) is greater than the root. Hence, it's a BST.



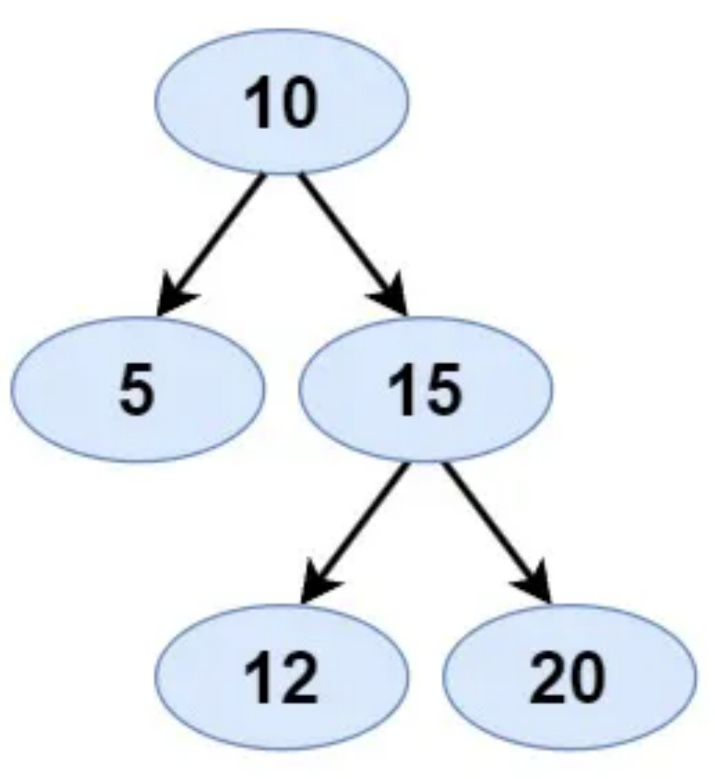
Example 2:

- **Input**: [5,7,3]
- **Expected Output**: false
- **Justification**: The left child of the root (7) is greater than the root, making it invalid.



Example 3:

- **Input**: [10,5,15,null,null,12,20]
- **Expected Output**: true
- **Justification**: Each subtree of the binary tree is a valid binary search tree. So, a whole binary tree is a valid binary search tree.



Solution

In essence, to validate if a given binary tree is a binary search tree (BST), we employ a recursive approach that checks the validity of each node by comparing its value with a permissible range. This range is determined by the node's ancestors, ensuring every node meets the BST property. Initially, the root node can take any value between negative infinity and positive infinity. As we traverse down the tree, we update this range based on the current node's value.

Detailed Breakdown:

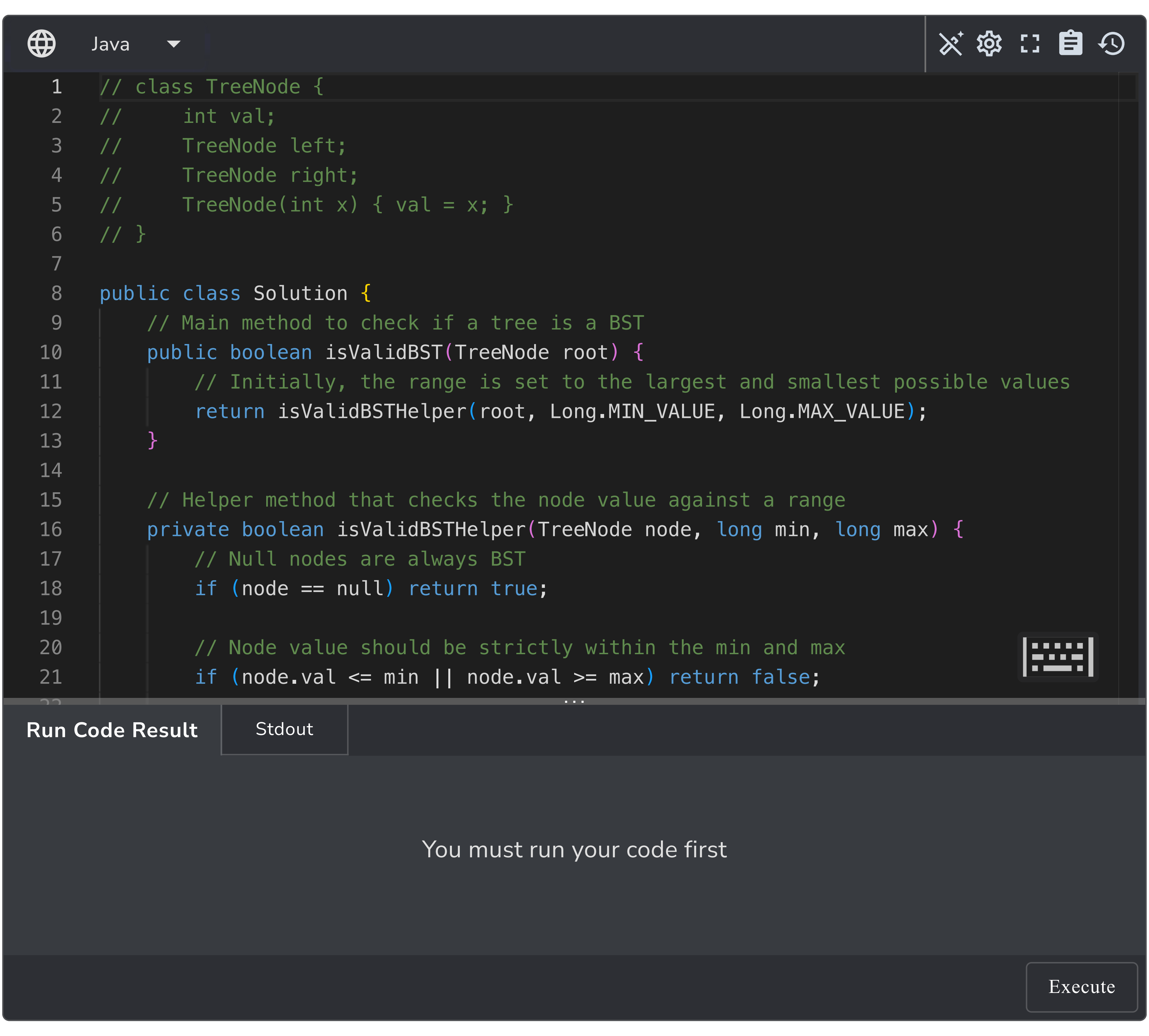
1. **Recursion**:
   - For each node in the binary tree, we validate its value against a permissible range. If the value does not lie within this range, the tree is not a BST.
   - The range for any node is influenced by its ancestors, ensuring that every node, even those deep in the tree, satisfies the BST condition.
2. **Base Case**:
   - If the node we are inspecting is null (i.e., we've reached a leaf node), we return true since a null subtree is always a BST by definition.
3. **Recursive Step**:
   - Compare the node's value against its permissible range. If it's not within the range, return false.
   - If it is within range, we recursively check the left and right children, but with updated permissible ranges.
4. **Implementation**:
   - Initially, the root node's range is set to (-Infinity, +Infinity).
   - For every left child, the upper limit of its permissible range is updated to the current node's value. Similarly, for every right child, the lower limit is updated to the current node's value.

Algorithm Walkthrough

For the tree [10,5,15,null,null,12,20]:

- Start with the root, range = (-Infinity, +Infinity)
  - Node 10 is within the range.
- Move to the left child, range = (-Infinity, 10)
  - Node 5 is within the range.
- Move to the right child of root, range = (10, +Infinity)
  - Node 15 is within the range.
- Move to the left child of 15, range = (10, 15)
  - Node 12 is within the range.
- Move to the right child of 15, range = (15, +Infinity)
  - Node 20 is within the range.
- return true, as the next left node is null.

Code



```
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {
    // Main method to check if a tree is a BST
    public boolean isValidBST(TreeNode root) {
        // Initially, the range is set to the largest and smallest possible values
        return isValidBSTHelper(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }

    // Helper method that checks the node value against a range
    private boolean isValidBSTHelper(TreeNode node, long min, long max) {
        // Null nodes are always BST
        if (node == null) return true;

        // Node value should be strictly within the min and max
        if (node.val <= min || node.val >= max) return false;
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(n) - We traverse each node once.
- **Space Complexity**: O(h) - The space is determined by the height of the tree due to recursive calls. In the worst case (skewed tree), it's O(n), while in the best case (balanced tree), it's O(log n).

← Previous
Validate Binary Search Tree (Medium)

Next →
Construct Binary Tree From Preorder And Inorder Traversal (Medium)

☐ Mark as Completed